

Adopt-a-Drain: Smart Drain Monitoring System

1. Problem Statement

Urban drains in many cities discharge untreated sewage and industrial waste directly into nearby rivers and water bodies. Due to the absence of real-time monitoring systems, pollution levels are often detected very late. This delay in response leads to:

- Increased water pollution
- Foul odor in surrounding areas
- Harm to aquatic life
- Long-term ecosystem damage

Currently, monitoring is mostly manual and periodic, making it inefficient and reactive instead of preventive.

2. Proposed Solution

The proposed system introduces a **low-cost IoT-based smart drain monitoring system** that continuously monitors key water quality parameters in real time.

The system:

- Measures water quality using multiple sensors
- Uploads data to a cloud platform
- Displays live readings on a dashboard
- Generates alerts when pollution crosses safe thresholds

This enables early detection and faster action by authorities and communities.

```

#include <OneWire.h>
#include <DallasTemperature.h>

// WiFi Credentials
const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";

// Pin Definitions
#define PH_PIN A0
#define TURBIDITY_PIN D1
#define ONE_WIRE_BUS D2
#define TRIG_PIN D5
#define ECHO_PIN D6
#define BUZZER D7

// Temperature Sensor Setup
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);

// Threshold Values
float phLow = 6.5;
float phHigh = 8.5;
int turbidityLimit = 700;
float tempLimit = 40.0;
float waterLevelLimit = 15.0;

void setup()
{
  Serial.begin(115200);

  pinMode(TURBIDITY_PIN, INPUT);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BUZZER, OUTPUT);

  sensors.begin();

  // WiFi Connection
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");

  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi Connected");
  Serial.println(WiFi.localIP());
}

void loop()
{
  // =====
  // pH Sensor Reading
  // =====
  int phValue = analogRead(PH_PIN);
  float voltage = phValue * (3.3 / 1023.0);
  float pH = 7 + ((2.5 - voltage) / 0.18);

  // =====
  // Turbidity Sensor
  // =====
  int turbidity = digitalRead(TURBIDITY_PIN);

  // =====
  // Temperature Sensor

```

```

// =====
sensors.requestTemperatures();
float temperature = sensors.getTempCByIndex(0);

// =====
// Ultrasonic Sensor
// =====
digitalWrite(TRIG_PIN, LOW);
delayMicroseconds(2);

digitalWrite(TRIG_PIN, HIGH);
delayMicroseconds(10);
digitalWrite(TRIG_PIN, LOW);

long duration = pulseIn(ECHO_PIN, HIGH);
float distance = duration * 0.034 / 2;

// =====
// Display Data
// =====
Serial.println("-----");
Serial.print("pH Value: ");
Serial.println(pH);

Serial.print("Turbidity: ");
Serial.println(turbidity);

Serial.print("Temperature: ");
Serial.println(temperature);

Serial.print("Water Level Distance: ");
Serial.println(distance);

// =====
// Alert Conditions
// =====
bool alert = false;

if (pH < pHLow || pH > pHHigh)
{
    Serial.println("ALERT: Unsafe pH Level!");
    alert = true;
}

if (turbidity == HIGH)
{
    Serial.println("ALERT: Dirty Water Detected!");
    alert = true;
}

if (temperature > tempLimit)
{
    Serial.println("ALERT: High Temperature!");
    alert = true;
}

if (distance < waterLevelLimit)
{
    Serial.println("ALERT: Water Level High!");
    alert = true;
}

// Buzzer Alert
if (alert)

```

3. System Architecture

The project is based on embedded systems and IoT principles. It consists of three main subsystems:

A. Sensing (Instrumentation)

The following sensors are used:

- **pH Sensor** - Measures acidity/alkalinity of water
- **Turbidity Sensor** - Detects water clarity (suspended particles)
- **Temperature Sensor** - Indicates abnormal thermal pollution
- **Ultrasonic Sensor** - Measures water level in drain

Each sensor converts physical parameters into electrical signals.

B. Processing (Controller Unit)

An **Arduino / NodeMCU** microcontroller is used to:

- Read analog/digital sensor data
- Process and filter readings
- Compare values with predefined safe thresholds
- Prepare data for cloud transmission

The controller acts as the brain of the system.

C. Communication s Actuation

- A **WiFi module** sends data to the cloud platform (e.g., ThingSpeak).
- If pollution exceeds safe limits, the system triggers:
 - SMS / App notification
 - Dashboard alert
 - Possible buzzer indication (optional)

Working Principle

1. The system remains powered ON continuously.
2. Sensors continuously monitor drain water parameters.
3. Data is collected by the microcontroller.
4. The controller uploads real-time readings to the cloud.
5. Dashboard displays live graphs of:
 - a. pH level
 - b. Turbidity
 - c. Temperature
 - d. Water level
6. If any parameter exceeds the safe threshold:
 - a. Alert is generated instantly.

This ensures real-time environmental monitoring.

4. Output and Results

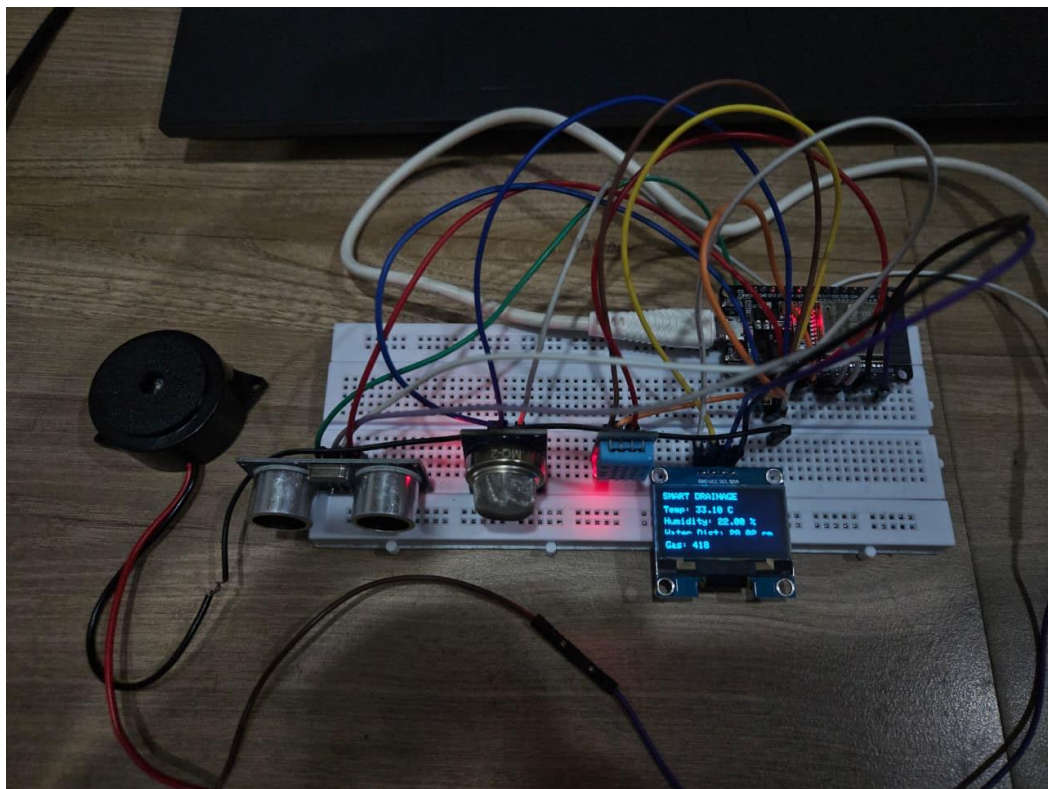
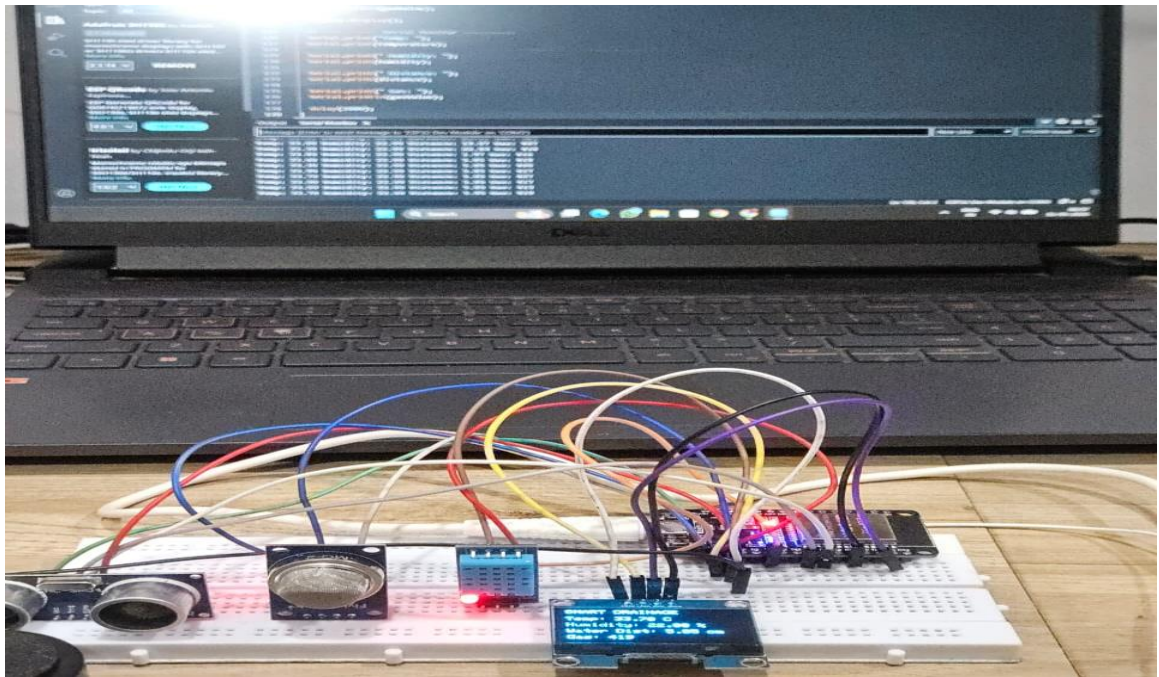
The prototype successfully demonstrates:

- Real-time monitoring of drain conditions
- Graphical visualization of pollution trends
- Instant alert generation
- Low implementation cost (₹4,000 - ₹6,000 approx.)

The system proves to be cost-effective and scalable.

5. Safety and Reliability Features

- Continuous automated monitoring
- Threshold-based alert system
- Battery/Solar power backup option



A screenshot of a 'Serial Monitor' window. The title bar says 'Output Serial Monitor X'. Below the title bar is a text input field with the placeholder 'Message (Enter to send message to 'ESP32 Dev Module' on 'COM3')'. To the right of the input field are two buttons: 'New Line' and '115200 baud'. The main area of the window displays a list of sensor readings, each on a new line. The readings are: Temp: 33.30 Humidity: 22.00 Distance: 4.56 Gas: 423; Temp: 33.90 Humidity: 22.00 Distance: 4.56 Gas: 415; Temp: 33.90 Humidity: 22.00 Distance: 4.56 Gas: 419; Temp: 33.80 Humidity: 22.00 Distance: 3.26 Gas: 399; Temp: 33.10 Humidity: 22.00 Distance: 3.26 Gas: 416; Temp: 33.40 Humidity: 22.00 Distance: 3.26 Gas: 423; Temp: 33.30 Humidity: 21.00 Distance: 3.26 Gas: 400; Temp: 33.00 Humidity: 23.00 Distance: 3.26 Gas: 403; Temp: 33.60 Humidity: 22.00 Distance: 3.26 Gas: 407; Temp: 33.40 Humidity: 22.00 Distance: 3.26 Gas: 401.

```
Temp: 33.30 Humidity: 22.00 Distance: 4.56 Gas: 423
Temp: 33.90 Humidity: 22.00 Distance: 4.56 Gas: 415
Temp: 33.90 Humidity: 22.00 Distance: 4.56 Gas: 419
Temp: 33.80 Humidity: 22.00 Distance: 3.26 Gas: 399
Temp: 33.10 Humidity: 22.00 Distance: 3.26 Gas: 416
Temp: 33.40 Humidity: 22.00 Distance: 3.26 Gas: 423
Temp: 33.30 Humidity: 21.00 Distance: 3.26 Gas: 400
Temp: 33.00 Humidity: 23.00 Distance: 3.26 Gas: 403
Temp: 33.60 Humidity: 22.00 Distance: 3.26 Gas: 407
Temp: 33.40 Humidity: 22.00 Distance: 3.26 Gas: 401
```

- Weather-resistant casing (recommended for real deployment)

The system is designed to operate with minimal maintenance.

6. Innovation and Future Scope

This project introduces:

- Community-driven drain adoption model
- Smart City compatible solution
- Scalable architecture for multiple drains

Future improvements may include:

- Solar-powered 24×7 autonomous operation
- Machine Learning for pollution prediction
- Integration with municipal dashboards
- Mobile application for citizen reporting

7. Conclusion

The Adopt-a-Drain Smart Monitoring System is a practical and affordable solution to urban water pollution monitoring. By combining IoT technology with community participation, the system enables real-time pollution tracking and faster intervention.

This project represents a small but impactful step toward cleaner rivers and smarter cities

